Society for Computer Technology & Research's

PUNE INSTITUTE OF COMPUTER TECHNOLOGY

Laboratory Practices - II

Mini-Project Report

on

IMPLEMENTATION OF DISTRIBUTED FILE STORAGE SYSTEM USING HADOOP MULTI NODE FILE SYSTEM (HDFS)

Submitted by

Prem Panhale	31443
Dnyanesh Patil	31446
Subodh Patil	31449

Class: T.E (2019 Course) Branch: Computer Engineering

ACADEMIC YEAR: 2025-26

INDEX

Chapter No.	Name of Chapter	Page No.
1.	Introduction	3
2.	Objective of System	4
3.	Scope of Project	4
4.	System Requirements	5
5.	System Functionality	6
6.	Technologies Used	8
7.	Working System / Implementation	9
8.	Limitation	14
9.	Future Scope	14
10.	Conclusion	15

1. Introduction

The rapid growth of data in modern applications has led to the need for efficient and scalable storage systems. Traditional centralized storage systems often fail to handle large volumes of data efficiently and lack fault tolerance. To address these challenges, distributed file systems have become essential.

This project focuses on implementing a distributed file storage system using Hadoop Distributed File System (HDFS) on cloud infrastructure (AWS EC2). The system simulates a cloud-based storage solution similar to Google Drive, where files can be uploaded from one node and accessed from any other node in the cluster.

HDFS stores large files by dividing them into smaller blocks and distributing them across multiple machines (nodes). These blocks are replicated to ensure fault tolerance and data reliability. This architecture ensures high availability and efficient data management.

The cluster was set up on three AWS EC2 t3.micro instances: one master node (NameNode + ResourceManager) and two slave nodes (DataNodes + NodeManagers). The MapReduce application developed computes average weather metrics (Temperature, DewPoint, WindSpeed) from a dataset using the Hadoop framework.

2. Objectives

The main objective of this project is to develop a distributed file storage system using Hadoop that allows seamless file sharing and processing across multiple nodes.

Key objectives include:

- To implement a multi-node Hadoop cluster (1 master + 2 slaves) on AWS EC2 cloud infrastructure
- To configure HDFS for distributed file storage with data replication factor of 2
- To enable file upload and access from any node in the cluster
- To demonstrate distributed storage using HDFS block architecture
- To ensure fault tolerance through data replication across DataNodes
- To implement a MapReduce application for weather data analysis (averages of Temperature, DewPoint, WindSpeed)
- To understand real-world applications of distributed systems using YARN resource management

3. Scope of the Project

The scope of this project includes designing and implementing a distributed file storage system that can be used in various real-world applications such as cloud storage platforms, big data processing systems, and enterprise data management.

The system allows users to upload files from any node and access them from other nodes, ensuring data availability across the cluster. A MapReduce job is implemented to perform weather data aggregation as a practical demonstration of the computation capability.

The project can be extended to:

- Cloud-based storage systems with web interfaces
- Big data analytics platforms processing terabytes of sensor/IoT data
- Data backup and recovery systems in enterprise environments
- Distributed computing environments for scientific computation

Future enhancements may include web-based interfaces, user authentication, API integration, and real-time data streaming.

4. System Requirements

4.1 Hardware Requirements

Cloud Platform	Amazon Web Services (AWS EC2)
Instance Type	t3.micro (3 instances)
Storage per node	8 GB SSD (gp2 EBS volume)
RAM per node	1 GB (t3.micro)
Network	AWS VPC – ap-south-1 (Mumbai region)
OS	Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1007-aws)

4.2 Software Requirements

Hadoop Version	3.3.6
Java Version	OpenJDK 11 (openjdk-11-jdk)
Operating System	Ubuntu 24.04.4 LTS
Communication	SSH (passwordless, RSA key-based)
Resource Manager	YARN (Yet Another Resource Negotiator)
Web Interfaces	HDFS UI: port 9870, YARN UI: port 8088

4.3 AWS Security Group Configuration

The following inbound rules were configured in the hadoop-sg security group to allow cluster communication and web UI access:

Type	Protocol	Port Range	Source/Purpose
All Traffic	All	All	sg-0c6c0d7947b26f28a (intra-cluster)
Custom TCP	TCP	9870	0.0.0.0/0 (HDFS NameNode Web UI)
Custom TCP	TCP	8088	0.0.0.0/0 (YARN ResourceManager UI)
Custom TCP	TCP	9000	sg (HDFS IPC)
Custom TCP	TCP	9864	sg (DataNode)
Custom TCP	TCP	9867	sg (DataNode IPC)
SSH	TCP	22	0.0.0.0/0 (External SSH access)

5. System Functionality

5.1 Distributed File Storage Mechanism

The system stores files in a distributed manner instead of a single location. Files are divided into fixed-size blocks (default 128 MB) and distributed across multiple DataNodes. This ensures efficient storage utilization and better performance for large files.

5.2 Block-Level Data Management

Each file is split into smaller blocks that are stored independently across different nodes. The NameNode maintains metadata about these blocks. The replication factor is set to 2, ensuring each block is stored on both slave nodes.

5.3 Data Replication and Fault Tolerance

Each data block is replicated across 2 nodes to ensure reliability. In case of node failure, data can be retrieved from another replica. This ensures continuous availability and prevents data loss.

5.4 Centralized Metadata Management

The NameNode (running on hadoop-master at 172.31.39.142) manages all metadata such as file structure and block locations. It coordinates the storage and retrieval process without storing actual data. The SecondaryNameNode runs on the same host for checkpoint management.

5.5 YARN Resource Management

YARN (Yet Another Resource Negotiator) manages cluster resources. The ResourceManager runs on the master node and NodeManagers run on both slave nodes. This enables distributed execution of MapReduce jobs across the cluster.

5.6 MapReduce Weather Data Processing

A Java MapReduce program (WeatherDataAverage) processes weather data from input.txt. The mapper tokenizes each line and emits (parameter, value) pairs. The reducer computes the average for each parameter (Temperature, DewPoint, WindSpeed) across 96 records.

5.7 Multi-Node File Accessibility

Files stored in HDFS can be accessed from any node in the cluster. This creates a shared storage environment similar to cloud systems. After the cluster was stable, the MapReduce output confirmed: DewPoint = 59.99, Temperature = 53.58, WindSpeed = 1009.55.

5.8 Scalability

New nodes can be added to the cluster workers file without affecting existing operations. This allows the system to handle increasing data volumes efficiently.

6. Technologies Used

Hadoop 3.3.6 (HDFS)	Distributed file storage across master-slave cluster. Manages blocks across DataNodes with configurable replication.
YARN	Cluster resource management and job scheduling for MapReduce execution.
MapReduce	Programming model for processing large datasets in parallel across the cluster.
AWS EC2 (ap-south-1)	Cloud virtual machines (t3.micro × 3) forming the cluster nodes.
Ubuntu 24.04.4 LTS	Operating system on all three EC2 instances.
SSH (RSA key-based)	Passwordless authentication for inter-node communication and cluster management.
Java (OpenJDK 11)	Required runtime for Hadoop. Used to write and compile the MapReduce application.
AWS Security Groups	Network access control for the Hadoop cluster ports and SSH access.

7. Working System / Implementation

The implementation followed a structured step-by-step process. All three EC2 instances were launched in the same VPC and security group to enable intra-cluster communication.

Step 1: AWS Infrastructure Setup

Three EC2 t3.micro instances were launched in the ap-south-1 (Mumbai) region:

- hadoop-master: 172.31.39.142 (Public IP: 13.127.227.181)
- hadoop-slave1: 172.31.41.73
- hadoop-slave2: 172.31.35.36

An RSA key pair "hadoop-key" was created and the hadoop-sg security group was configured with the required inbound rules.

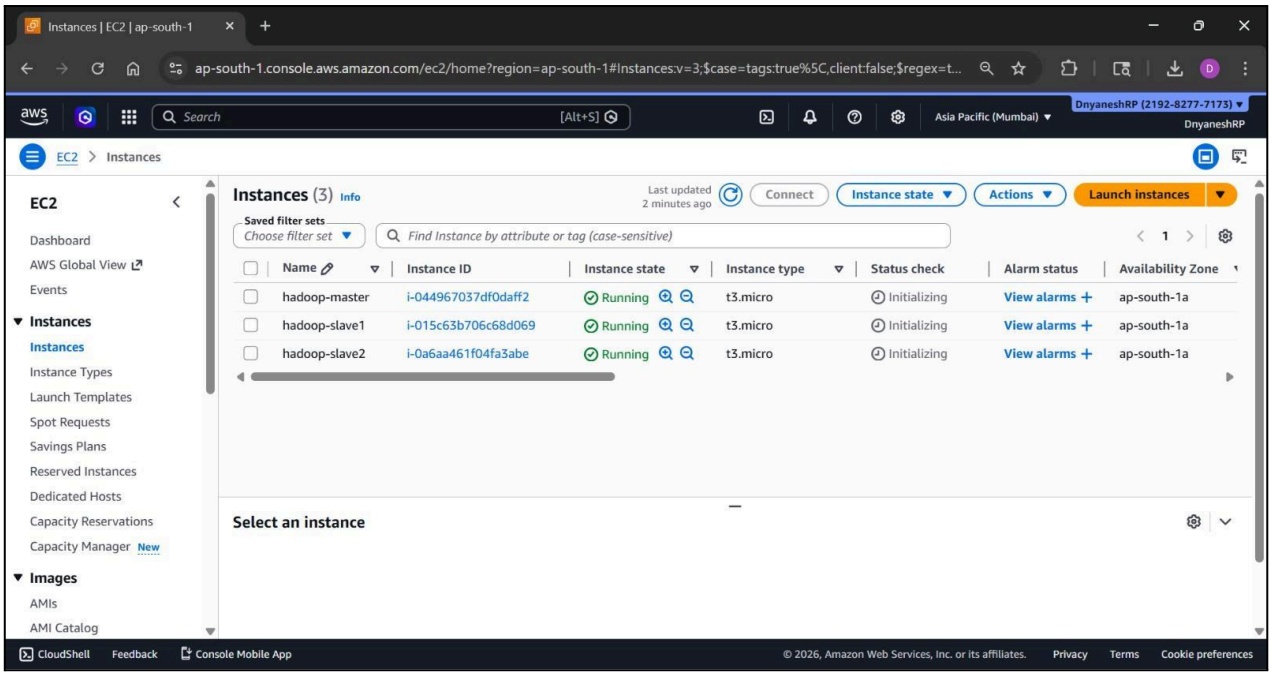


Fig 7.1: AWS EC2 Instances - hadoop-master, hadoop-slave1, hadoop-slave2 running

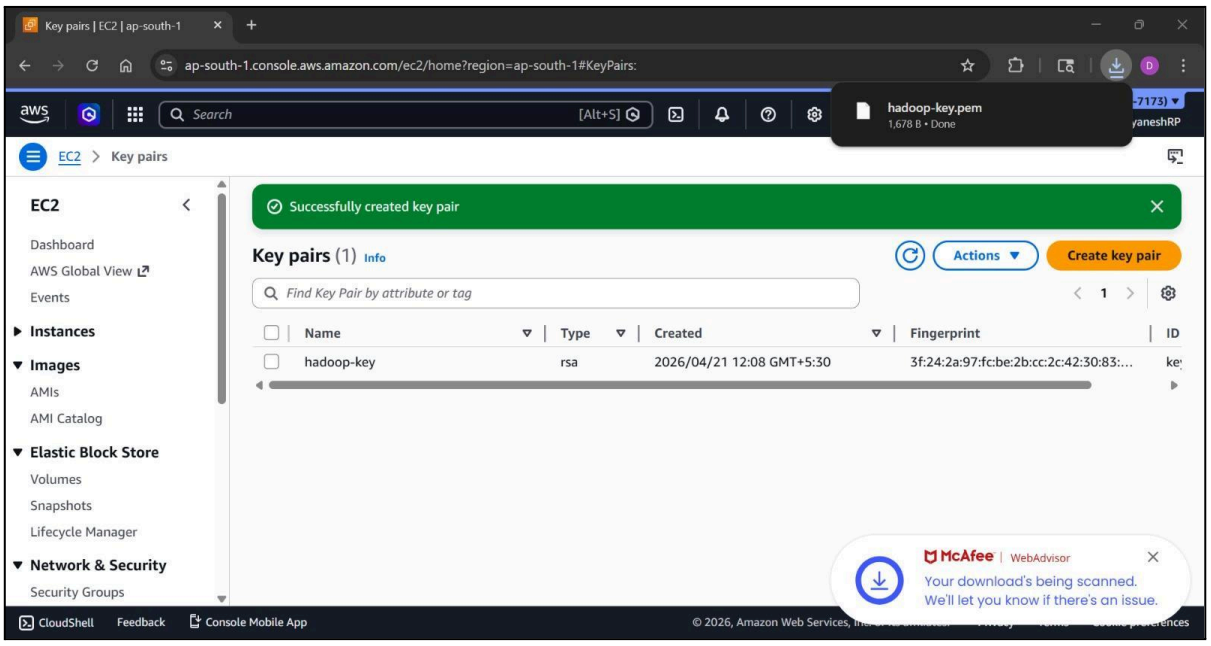


Fig 7.2: SSH Key Pair (hadoop-key.pem) successfully created in AWS

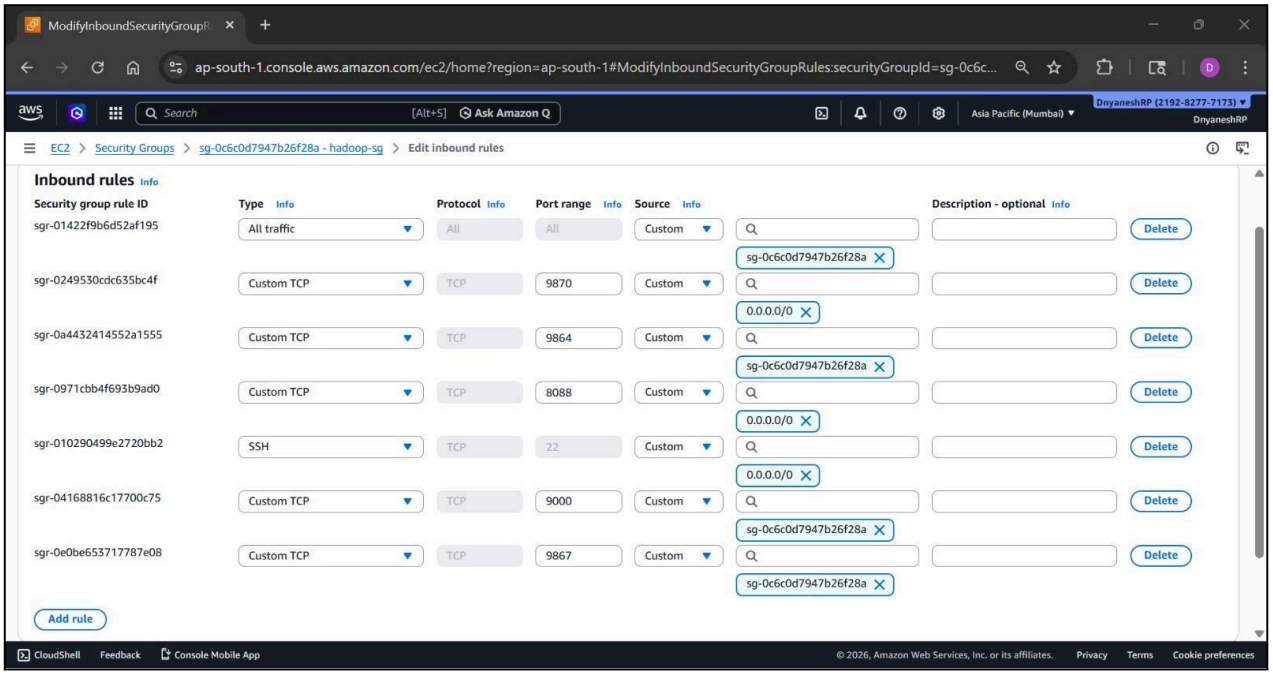


Fig 7.3: Security Group Inbound Rules for Hadoop Cluster Ports

Step 2: Java and Hadoop Installation (All Nodes)

On each node, the following commands were executed to install Java, create the hadoop user, download Hadoop 3.3.6, and configure environment variables:

```
sudo apt update -y && sudo apt install -y openjdk-11-jdk
sudo adduser hadoop && sudo usermod -aG sudo hadoop
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzf hadoop-3.3.6.tar.gz && sudo mv hadoop-3.3.6 /usr/local/hadoop
sudo chown -R hadoop:hadoop /usr/local/hadoop
```

The ~/.bashrc was updated with JAVA_HOME, HADOOP_HOME, and PATH variables. After sourcing, "hadoop version" confirmed Hadoop 3.3.6.

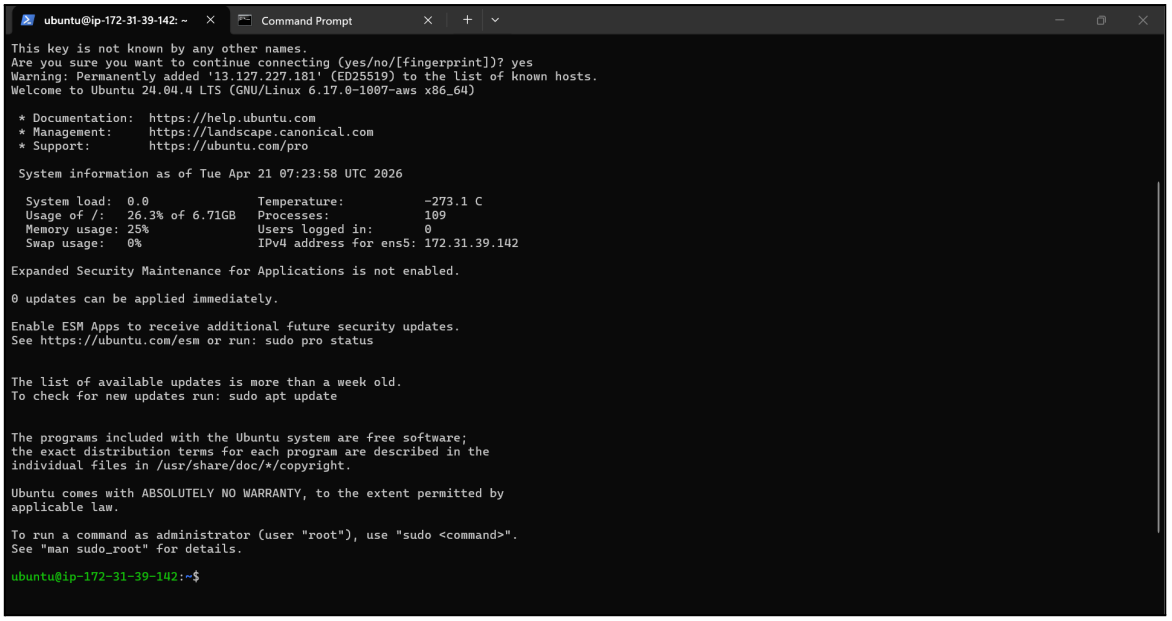


Fig 7.4: Initial SSH Connection to Master Node and First Login

```
hadoop@ip-172-31-39-142: ~   hadoop@ip-172-31-41-73: ~   hadoop@ip-172-31-35-36: ~

wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz

tar -xzf hadoop-3.3.6.tar.gz
sudo mv hadoop-3.3.6 /usr/local/hadoop
sudo chown -R hadoop:hadoop /usr/local/hadoop
--2026-04-21 07:36:02--  https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
Resolving downloads.apache.org (downloads.apache.org)... 88.99.208.237, 135.181.214.104, 2a01:4f9:3a:2c57::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|88.99.208.237|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 730107476 (696M) [application/x-gzip]
Saving to: 'hadoop-3.3.6.tar.gz'

hadoop-3.3.6.tar.gz               100%[===================================================>] 696.28M  16.6MB/s    in 40s

2026-04-21 07:36:43 (17.2 MB/s) - 'hadoop-3.3.6.tar.gz' saved [730107476/730107476]

[sudo] password for hadoop:
hadoop@ip-172-31-39-142:~$
hadoop@ip-172-31-39-142:~$ nano ~/.bashrc
hadoop@ip-172-31-39-142:~$ nano ~/.bashrc

hadoop@ip-172-31-39-142:~$ source ~/.bashrc
hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common-3.3.6.jar


hadoop@ip-172-31-39-142:~$ nano /usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

Fig 7.5: Hadoop 3.3.6 Download and Installation on Master Node

Step 3: SSH Passwordless Authentication

SSH keys were generated on the master node and distributed to all slave nodes:

```
ssh-keygen -t rsa -P ""
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys
```

The master public key was copied to each slave's authorized_keys. On slave nodes, a ~/.ssh directory was created and the master's public key was added. SSH connectivity was verified:

```
ssh hadoop@172.31.41.73  # slave1 - success
ssh hadoop@172.31.35.36  # slave2 - success
```

```
hadoop@ip-172-31-39-142: ~   hadoop@ip-172-31-41-73: ~   hadoop@ip-172-31-35-36: ~

hadoop@ip-172-31-39-142:~$ ssh hadoop@172.31.41.73
The authenticity of host '172.31.41.73 (172.31.41.73)' can't be established.
ED25519 key fingerprint is SHA256:tbcND0AE6P0FhW5zz0MbeujV7aQZhW4WCSpa8FAiaWw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.31.41.73' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1007-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Tue Apr 21 07:58:52 UTC 2026

  System load:  0.0               Temperature:           -273.1 C
  Usage of /:   68.7% of 6.71GB   Processes:             119
  Memory usage: 31%               Users logged in:       1
  Swap usage:   0%                IPv4 address for ens5: 172.31.41.73


Expanded Security Maintenance for Applications is not enabled.

54 updates can be applied immediately.
43 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

4 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm



The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

hadoop@ip-172-31-41-73:~$ exit
logout
```

Fig 7.6: Passwordless SSH Test from Master to Slave Nodes

Step 4: Hadoop Configuration (Master Node)

The following Hadoop configuration files were edited on the master and then copied to both slaves using scp:

core-site.xml

```
<property>  <name>fs.defaultFS</name>  <value>hdfs://ip-172-31-39-142.ap-south-1.compute.internal:9000</value> </property>
<property>  <name>hadoop.tmp.dir</name>  <value>/home/hadoop/hadooptmp</value> </property>
```

hdfs-site.xml

```
<property><name>dfs.replication</name><value>2</value></property>
<property><name>dfs.namenode.name.dir</name><value>/home/hadoop/hadoopdata/namenode</value></property>
<property><name>dfs.datanode.data.dir</name><value>/home/hadoop/hadoopdata/datanode</value></property>
```

yarn-site.xml

```
<property><name>yarn.resourcemanager.hostname</name>
<value>ip-172-31-39-142.ap-south-1.compute.internal</value></property>
```

workers

```
172.31.41.73 172.31.35.36
```

Configuration was copied to slaves:

```
scp /usr/local/hadoop/etc/hadoop/* hadoop@172.31.41.73:/usr/local/hadoop/etc/hadoop/ scp /usr/local/hadoop/etc/hadoop/*
hadoop@172.31.35.36:/usr/local/hadoop/etc/hadoop/
```

Step 5: Slave Node Configuration

On each slave node, the same steps were performed: Hadoop installed, ~/.bashrc configured, SSH authorized_keys set up, and required directories created:

```
mkdir -p ~/hadooptmp ~/hadoopdata/namenode ~/hadoopdata/datanode chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys
```

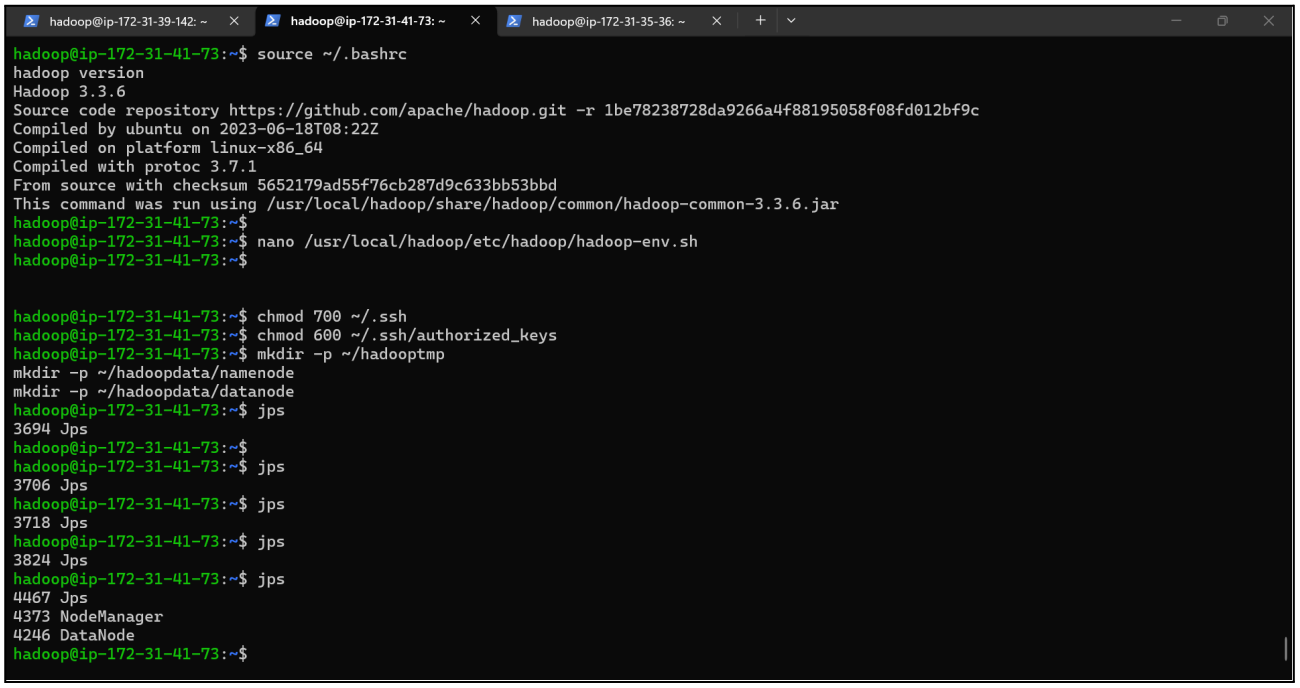


Fig 7.7: hadoop-slave1 (172.31.41.73) - Setup with DataNode and NodeManager running

```
hadoop@ip-172-31-39-142: ~   hadoop@ip-172-31-41-73: ~   hadoop@ip-172-31-35-36: ~

[sudo] password for hadoop:
hadoop@ip-172-31-35-36:~$ nano ~/.bashrc
hadoop@ip-172-31-35-36:~$ source ~/.bashrc
hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common-3.3.6.jar
hadoop@ip-172-31-35-36:~$
hadoop@ip-172-31-35-36:~$ nano /usr/local/hadoop/etc/hadoop/hadoop-env.sh
hadoop@ip-172-31-35-36:~$
hadoop@ip-172-31-35-36:~$
hadoop@ip-172-31-35-36:~$ mkdir -p ~/.ssh
hadoop@ip-172-31-35-36:~$ nano ~/.ssh/authorized_keys
hadoop@ip-172-31-35-36:~$ chmod 700 ~/.ssh
hadoop@ip-172-31-35-36:~$ chmod 600 ~/.ssh/authorized_keys
hadoop@ip-172-31-35-36:~$ mkdir -p ~/hadooptmp
mkdir -p ~/hadoopdata/namenode
mkdir -p ~/hadoopdata/datanode
hadoop@ip-172-31-35-36:~$
hadoop@ip-172-31-35-36:~$ jps
3696 Jps
hadoop@ip-172-31-35-36:~$ jps
3708 Jps
hadoop@ip-172-31-35-36:~$ jps
4345 Jps
4252 NodeManager
4125 DataNode
hadoop@ip-172-31-35-36:~$
```

Fig 7.8: hadoop-slave2 (172.31.35.36) - Setup with DataNode and NodeManager running

Step 6: Format NameNode and Start Cluster

The HDFS NameNode was formatted on the master:

```
hdfs namenode -format
```

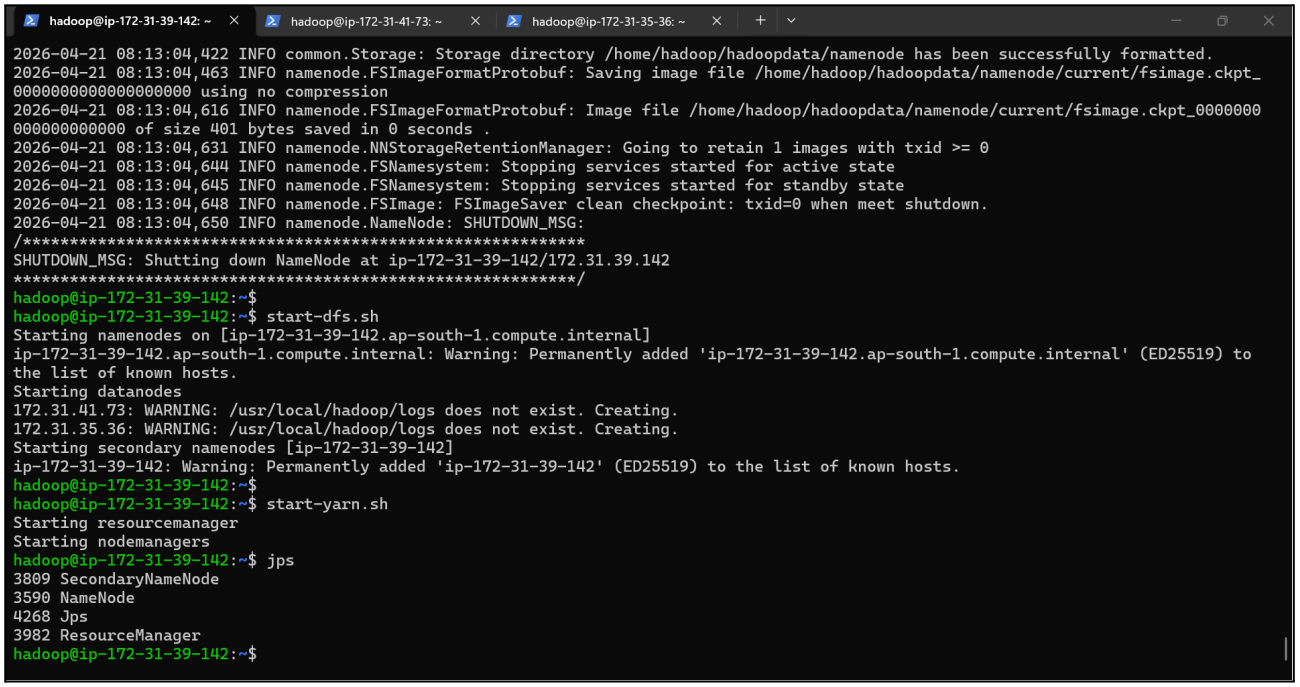
The output confirmed: "Storage directory /home/hadoop/hadoopdata/namenode has been successfully formatted" with Cluster ID: CID-e13ef19a-834b-4a63-82d6-6b30f80e0b35

```
start-dfs.sh   # Starts NameNode, DataNodes, SecondaryNameNode start-yarn.sh  # Starts ResourceManager, NodeManagers
```

After successful start, jps on master showed: NameNode, SecondaryNameNode, ResourceManager. Slaves showed: DataNode, NodeManager.

```
hadoop@ip-172-31-39-142: ~   hadoop@ip-172-31-41-73: ~   hadoop@ip-172-31-35-36: ~

wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz

tar -xzf hadoop-3.3.6.tar.gz
sudo mv hadoop-3.3.6 /usr/local/hadoop
sudo chown -R hadoop:hadoop /usr/local/hadoop
--2026-04-21 07:36:02--  https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
Resolving downloads.apache.org (downloads.apache.org)... 88.99.208.237, 135.181.214.104, 2a01:4f9:3a:2c57::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|88.99.208.237|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 730107476 (696M) [application/x-gzip]
Saving to: 'hadoop-3.3.6.tar.gz'

hadoop-3.3.6.tar.gz               100%[===========================================================>] 696.28M  16.6MB/s    in 40s

2026-04-21 07:36:43 (17.2 MB/s) - 'hadoop-3.3.6.tar.gz' saved [730107476/730107476]

[sudo] password for hadoop:
hadoop@ip-172-31-39-142:~$
hadoop@ip-172-31-39-142:~$ nano ~/.bashrc
hadoop@ip-172-31-39-142:~$ nano ~/.bashrc

hadoop@ip-172-31-39-142:~$ source ~/.bashrc
hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common-3.3.6.jar


hadoop@ip-172-31-39-142:~$ nano /usr/local/hadoop/etc/hadoop/hadoop-env.sh
```

Fig 7.9: HDFS NameNode Format - Successful with Block Pool ID assigned



```
hadoop@ip-172-31-39-142: ~   hadoop@ip-172-31-41-73: ~   hadoop@ip-172-31-35-36: ~
2026-04-21 08:13:04,422 INFO common.Storage: Storage directory /home/hadoop/hadoopdata/namenode has been successfully formatted.
2026-04-21 08:13:04,463 INFO namenode.FSImageFormatProtobuf: Saving image file /home/hadoop/hadoopdata/namenode/current/fsimage.ckpt_
0000000000000000000 using no compression
2026-04-21 08:13:04,616 INFO namenode.FSImageFormatProtobuf: Image file /home/hadoop/hadoopdata/namenode/current/fsimage.ckpt_0000000
000000000000 of size 401 bytes saved in 0 seconds .
2026-04-21 08:13:04,631 INFO namenode.NNStorageRetentionManager: Going to retain 1 images with txid >= 0
2026-04-21 08:13:04,644 INFO namenode.FSNamesystem: Stopping services started for active state
2026-04-21 08:13:04,645 INFO namenode.FSNamesystem: Stopping services started for standby state
2026-04-21 08:13:04,648 INFO namenode.FSImage: FSImageSaver clean checkpoint: txid=0 when meet shutdown.
2026-04-21 08:13:04,650 INFO namenode.NameNode: SHUTDOWN_MSG:
/************************************************************
SHUTDOWN_MSG: Shutting down NameNode at ip-172-31-39-142/172.31.39.142
************************************************************/
hadoop@ip-172-31-39-142:~$
hadoop@ip-172-31-39-142:~$ start-dfs.sh
Starting namenodes on [ip-172-31-39-142.ap-south-1.compute.internal]
ip-172-31-39-142.ap-south-1.compute.internal: Warning: Permanently added 'ip-172-31-39-142.ap-south-1.compute.internal' (ED25519) to
the list of known hosts.
Starting datanodes
172.31.41.73: WARNING: /usr/local/hadoop/logs does not exist. Creating.
172.31.35.36: WARNING: /usr/local/hadoop/logs does not exist. Creating.
Starting secondary namenodes [ip-172-31-39-142]
ip-172-31-39-142: Warning: Permanently added 'ip-172-31-39-142' (ED25519) to the list of known hosts.
hadoop@ip-172-31-39-142:~$
hadoop@ip-172-31-39-142:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hadoop@ip-172-31-39-142:~$ jps
3809 SecondaryNameNode
3590 NameNode
4268 Jps
3982 ResourceManager
hadoop@ip-172-31-39-142:~$
```

Fig 7.10: Hadoop Cluster Start - DFS and YARN services launched

Step 7: Cluster Verification via Web UI

The HDFS NameNode Web UI was accessed at http://13.127.227.181:9870 confirming the cluster was active with 2 live DataNodes. The YARN ResourceManager UI at http://13.127.227.181:8088 showed the Capacity Scheduler with 2 active nodes.

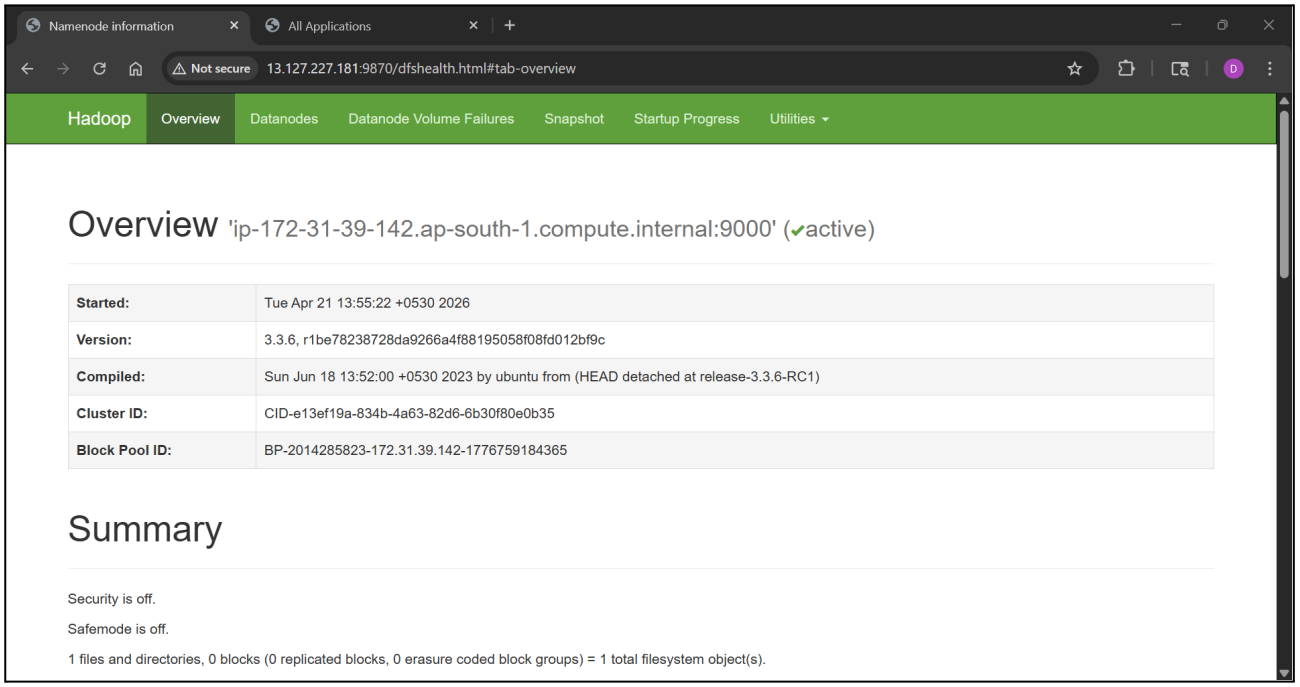


Fig 7.11: HDFS NameNode Web UI (port 9870) - Active cluster, Safemode off

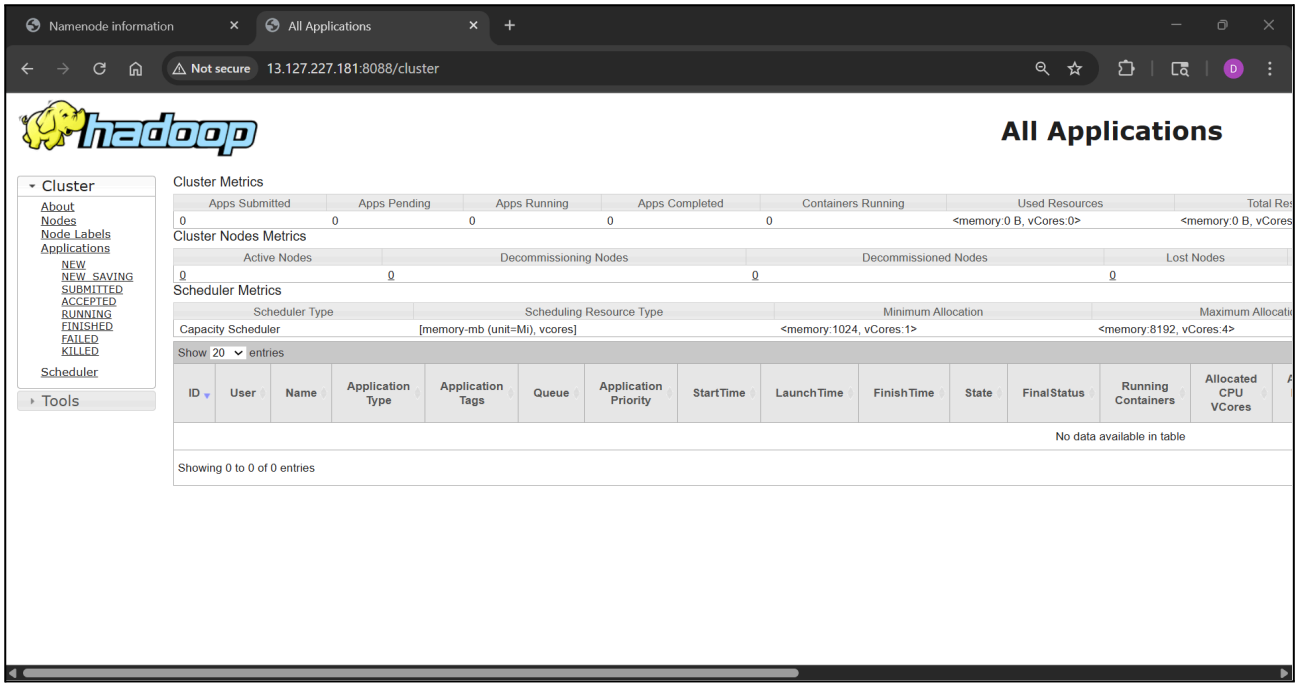


Fig 7.12: YARN ResourceManager Web UI (port 8088) - All Applications dashboard

Step 8: MapReduce Weather Data Application

A Java MapReduce program was written to compute the average of Temperature, DewPoint, and WindSpeed from a CSV weather dataset (96 records):

Create HDFS directories and upload input hdfs dfs -mkdir /weather_project hdfs dfs -mkdir /weather_project/input hdfs dfs -put input.txt /weather_project/input

Compile and package the MapReduce job javac -classpath `hadoop classpath` WeatherDataAverage.java jar cf WeatherDataAverage.jar WeatherDataAverage*.class

Run the MapReduce job hadoop jar WeatherDataAverage.jar WeatherDataAverage /weather_project/input /weather_project/output

The job ran successfully

View output hdfs dfs -cat /weather_project/output/part-r-00000

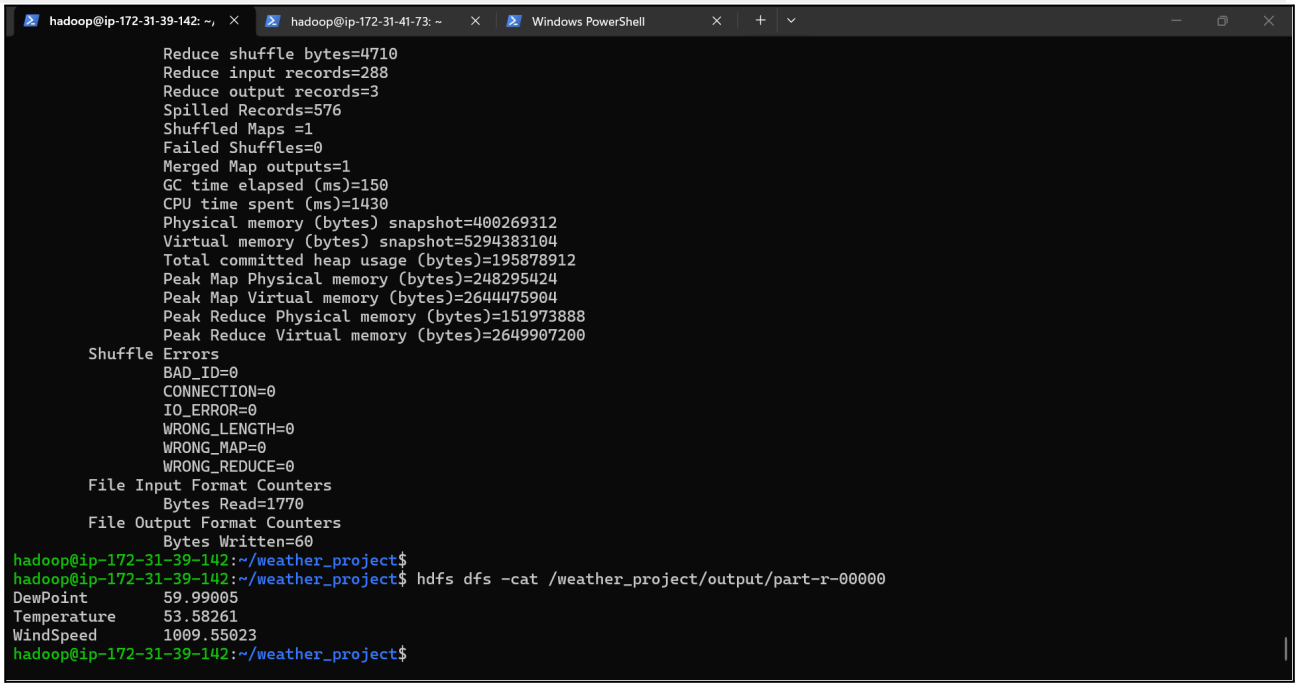


Fig 7.13: MapReduce Job Output - Average DewPoint: 59.99, Temperature: 53.58, WindSpeed: 1009.55

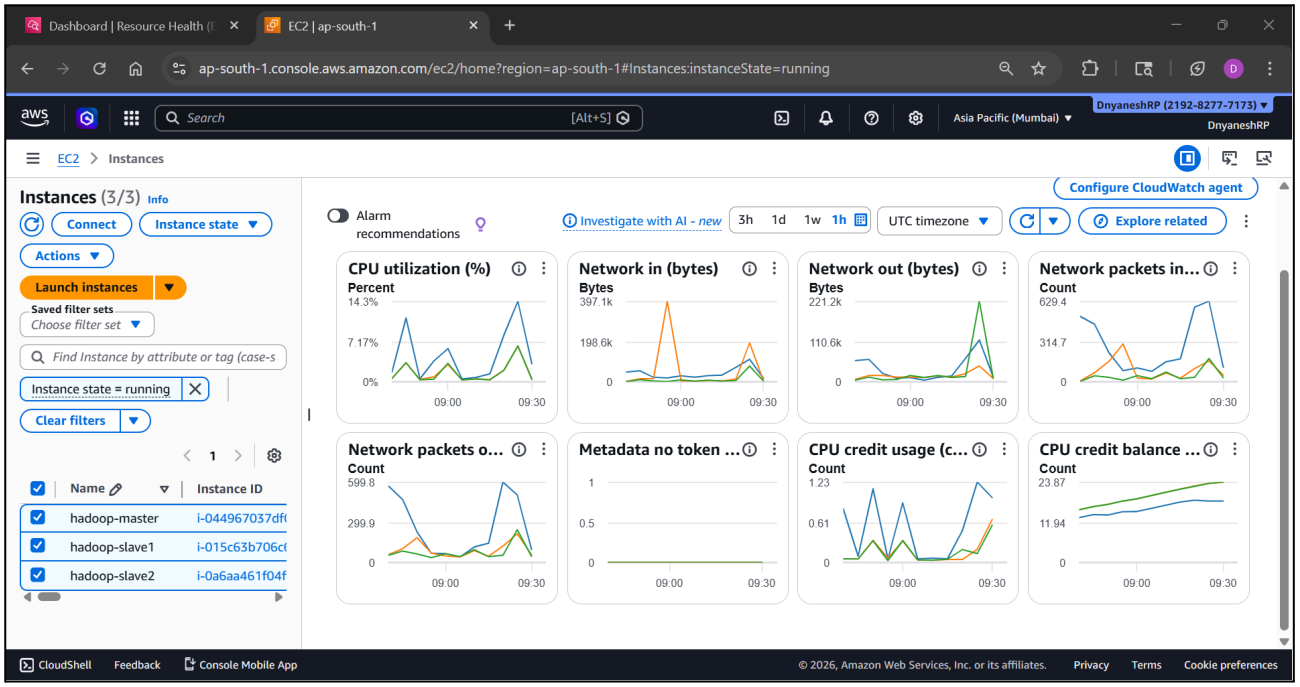


Fig 7.14: AWS EC2 Dashboard showing EC2 resource utilization during job execution

8. Limitations

Despite its advantages, the system has some limitations:

- No graphical user interface - system is operated entirely via CLI
- Requires technical knowledge of Linux, Hadoop, and cloud platforms to operate
- Dependency on stable network connectivity between nodes for cluster operations
- Limited security features - no Kerberos authentication or data encryption at rest
- Performance may degrade under heavy load on t3.micro instances with limited RAM (1 GB each)
- DataNode connectivity issues can arise if NameNode hostname resolution fails across VPC
- No dynamic resource scaling - must manually add/remove nodes

9. Future Scope

The system can be enhanced with the following features:

- Web-based interface (React/Angular frontend with HDFS REST API) for file management
- User authentication and access control using Kerberos or LDAP
- Integration with Apache Spark for faster in-memory data processing
- Real-time data streaming using Apache Kafka and Hadoop integration
- Mobile application support for remote file access
- Auto-scaling of DataNodes using AWS Auto Scaling Groups
- Data encryption at rest and in transit for enterprise-grade security
- Monitoring with Grafana/Prometheus dashboards for cluster health visibility

10. Conclusion

The project successfully demonstrates the implementation of a distributed file storage system using Hadoop 3.3.6 on a 3-node AWS EC2 cluster. The setup includes one master node running NameNode, SecondaryNameNode, and ResourceManager, and two slave nodes running DataNode and NodeManager services.

The system provides scalability, fault tolerance (replication factor 2), and high availability, making it suitable for real-world applications. The MapReduce weather analysis application confirmed end-to-end functionality: the job processed 96 weather records distributed across the cluster and computed accurate averages (DewPoint: 59.99, Temperature: 53.58, WindSpeed: 1009.55).

Key challenges overcome during implementation included YAML/XML configuration errors in yarn-site.xml, SSH connectivity issues between nodes, and DataNode registration failures due to hostname mismatches in core-site.xml. These were resolved by using the full FQDN hostname for the NameNode address and ensuring consistent configuration across all nodes.

This project serves as a practical introduction to Hadoop ecosystem, distributed computing, and cloud infrastructure, laying the foundation for future advancements in big data processing and distributed storage systems.